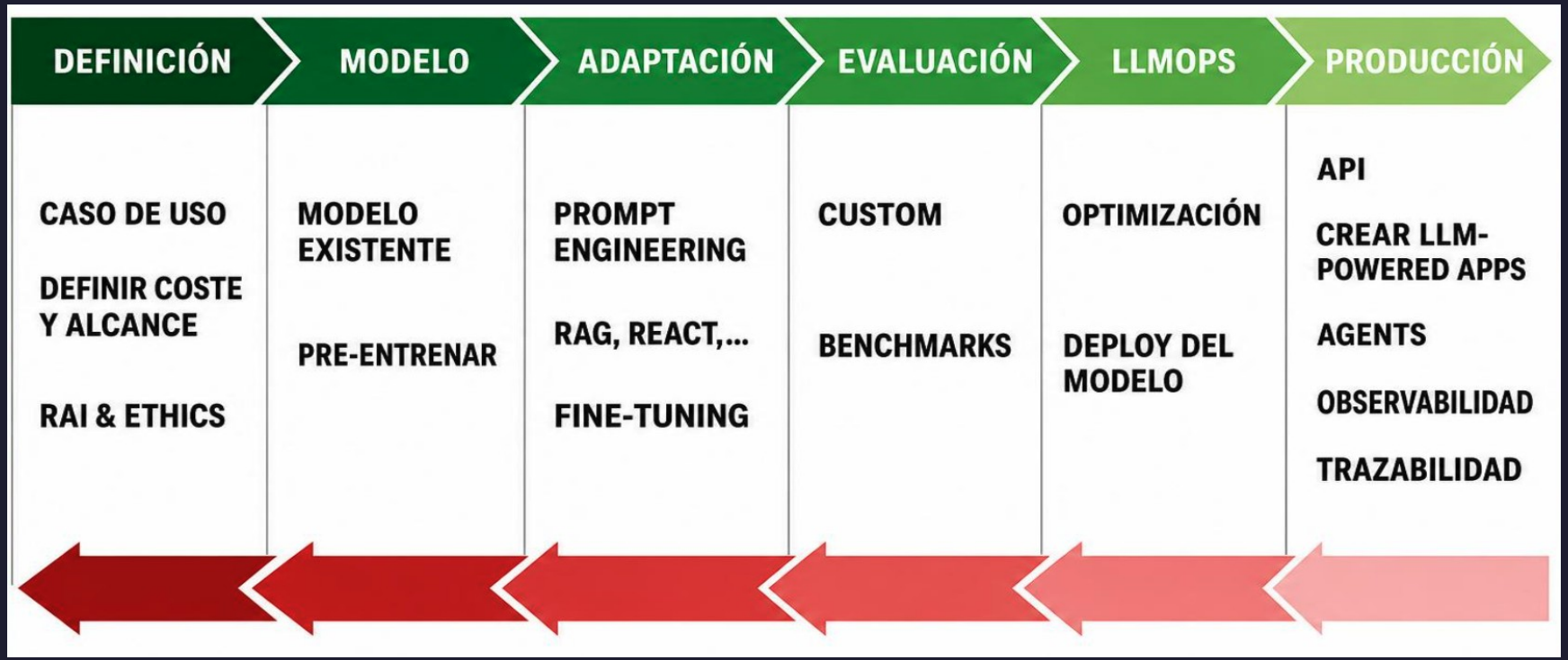
keep coding

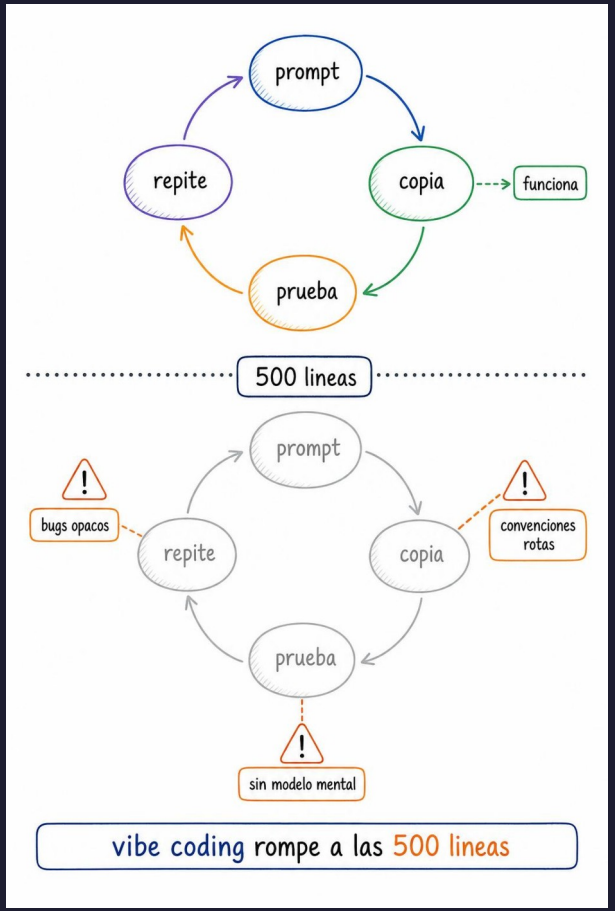
AI ENGINEERING

Spec-Driven Development


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

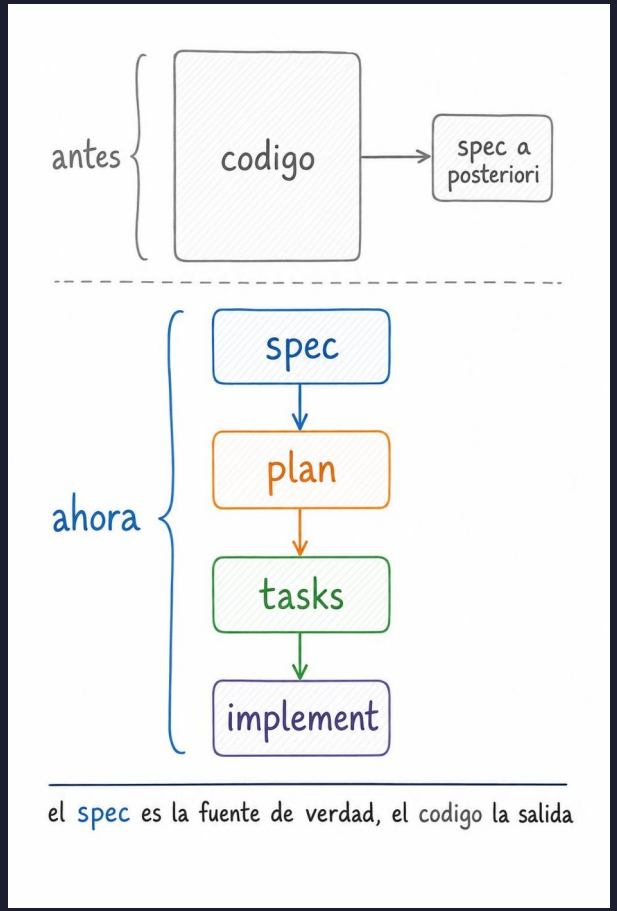
El problema del vibe coding

- Funciona increiblemente bien hasta unas 500 líneas
- Despues pierdes el modelo mental del código
- El contexto del LLM no cabe el repo entero
- Si hay bug, no sabes ni por donde empezar



La inversion · spec primero

- Spec primero, código despues
- Pipeline: Spec -> Plan -> Tasks -> Implement
- El spec es la fuente de verdad, el código es su salida
- Lo usan GitHub Spec-Kit, Kiro, Superpowers, Antigravity



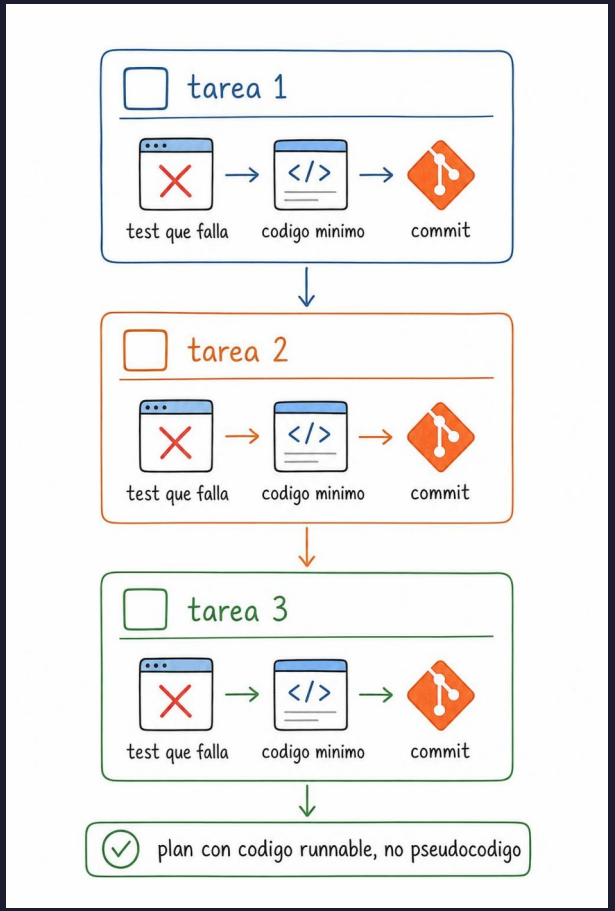
Anatomia de un spec

- Objetivo · que estamos construyendo y por qué
- Requisitos · funcionales y no funcionales
- Contratos · APIs, modelos de datos, interfaces
- Criterios de aceptacion · como sabemos que esta hecho
- Lo que queda fuera · igual de importante que lo que entra



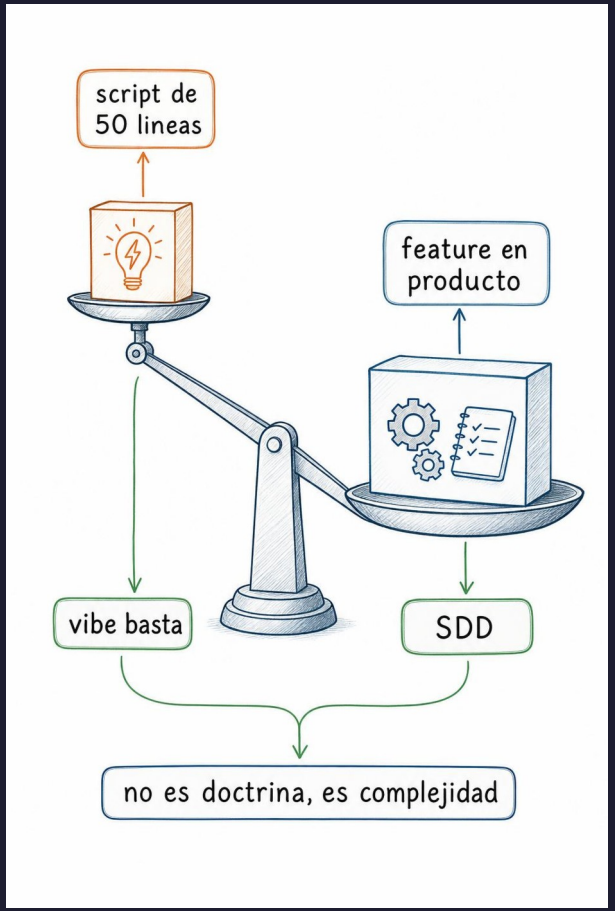
El plan · tareas con código real

- Tareas atomicas: una tarea es un commit
- Test que falla primero, despues la implementación mínima
- Código completo en el plan, no pseudocodigo (Superpowers)
- Cada tarea con criterio de aceptacion verificable



Cuando SDD y cuando no

- Script de 50 líneas: vibe basta
- Feature en producto: SDD obligatorio
- Refactor transversal: SDD obligatorio
- La frontera no es ideologica, es de complejidad




keep coding